

Azure and GCP Site-to-Site VPN with BGP and Cross-Cloud Private DNS

This guide explains how an Azure VNet ([10.10.0.0/16](#)) and a GCP VPC ([10.20.0.0/16](#)) are connected with a site-to-site IPsec VPN, how BGP automatically shares routes between them, and how virtual machines in each cloud find each other by **private DNS names** instead of IP addresses.

It uses Azure DNS Private Resolver (inbound subnet [10.10.50.0/28](#), outbound subnet [10.10.51.0/28](#)) and GCP Cloud DNS forwarding, walks through every DNS lookup step by step with diagrams, and ends with a practical troubleshooting guide and all the Azure CLI and gcloud commands used.

Author: Aslam Chandio

LinkedIn: [linkedin.com/in/aslam-chandio](https://www.linkedin.com/in/aslam-chandio)

1. A simple way to think about it

Imagine each cloud is an office with its own **reception desk** (its built-in DNS). Each desk has a phone book for its own office only.

- When someone asks about a person in the **other** office, the desk has a note saying: "For those names, call the other office's reception."
- Each office also has a **visitor phone line** that the other office can call.
- The **VPN tunnel** is the private phone line between the two offices.

Everything in this document is just that idea, with real names and IP addresses.

2. What lives where

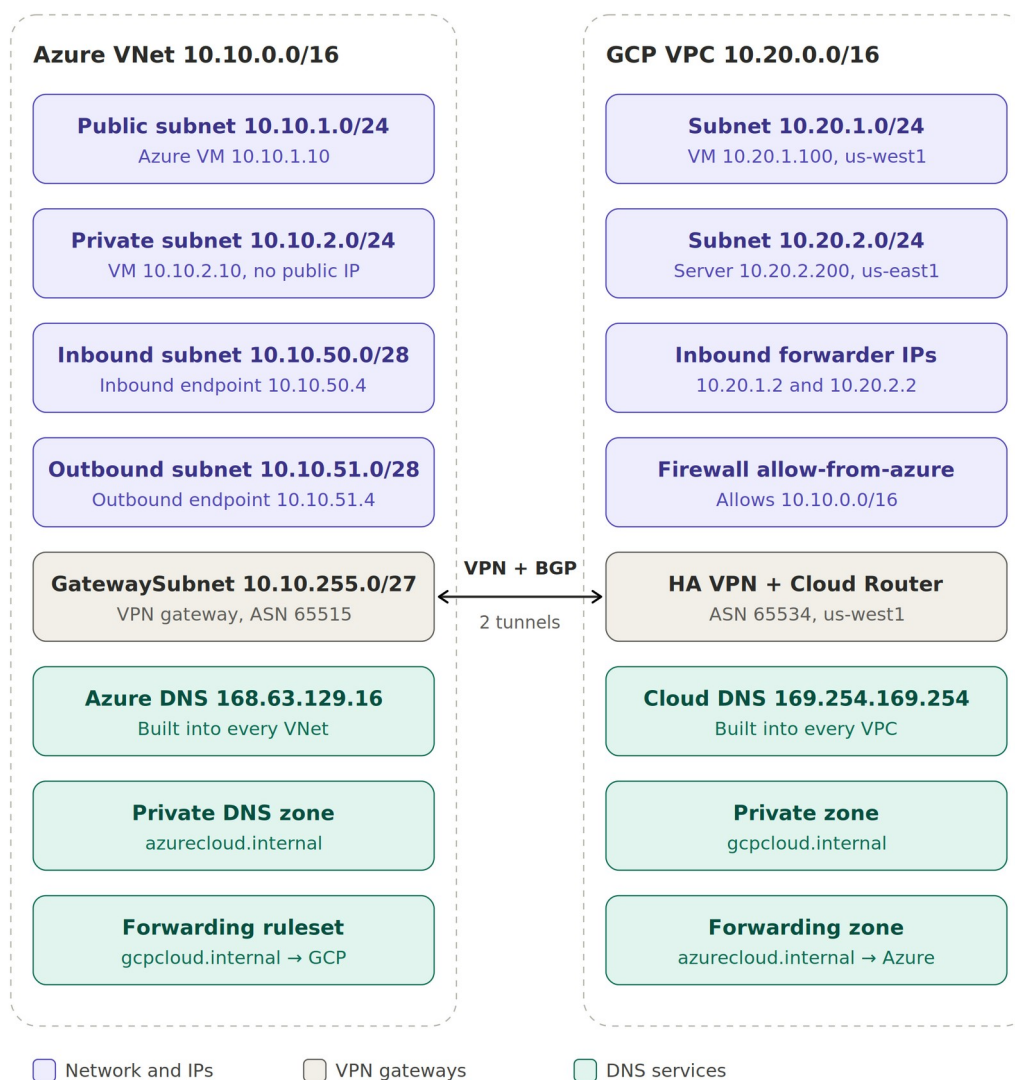


Diagram 1: Network layout of both clouds

Azure side (VNet 10.10.0.0/16)

- **Public subnet 10.10.1.0/24:** for VMs that can have public IPs and internet access. Example: the VM at 10.10.1.10, which has the name `public.azurecloud.internal`.
- **Private subnet 10.10.2.0/24:** a second subnet created as an Azure **private subnet** (no default outbound internet access). Example: the VM at 10.10.2.10 (`private-vm.azurecloud.internal`). It is inside 10.10.0.0/16, so it is reached over the VPN with no extra routing work.
- **Inbound subnet 10.10.50.0/28:** used only by the DNS resolver's inbound endpoint, which gets the IP 10.10.50.4. This is Azure's visitor phone line: GCP sends its questions here.
- **Outbound subnet 10.10.51.0/28:** used only by the outbound endpoint. When Azure needs to ask GCP something, the question leaves from here (for example from 10.10.51.4).
- **GatewaySubnet 10.10.255.0/27:** holds the Azure VPN gateway, which builds the tunnel to GCP and talks BGP using ASN 65515.
- **Azure DNS 168.63.129.16:** the reception desk. It exists automatically in every VNet, and every Azure VM uses it by default.

- **Private DNS zone `azurecloud.internal`**: Azure's phone book, holding records like `public.azurecloud.internal → 10.10.1.10`. It only works for VNets it is linked to.
- **Forwarding ruleset**: the note on the desk that says "questions about `gcpcloud.internal`, send to GCP."

Rules for the two resolver subnets:

- They must be **at least /28** (16 addresses).
- They must be **delegated** to `Microsoft.Network/dnsResolvers`, so nothing else (no VMs) can be placed in them.
- Azure reserves the first 4 addresses of every subnet, so `.4` is the first usable IP. That is why the inbound endpoint is `10.10.50.4`.

GCP side (VPC 10.20.0.0/16)

- **Subnet `10.20.1.0/24` (region `us-west1`)**: in the **same region** as the HA VPN and Cloud Router. Example: the VM `10.20.1.100` (`private.gcpcloud.internal`).
- **Subnet `10.20.2.0/24` (region `us-east1`)**: in a **different region** from the VPN. Example: the server `10.20.2.200` (`public.gcpcloud.internal`). It reaches Azure only through **global routing** (section 6).
- **Firewall rule `allow-from-azure`**: allows traffic from `10.10.0.0/16`, including DNS queries from Azure's outbound endpoint. GCP firewall rules are global, so it covers both regions.
- **Inbound forwarder IPs**: GCP's visitor phone line. GCP creates one automatically in each subnet when the inbound DNS policy is turned on, which is why there are two: `10.20.1.2` (`us-west1`) and `10.20.2.2` (`us-east1`). The `.2` values are examples; use the ones from `gcloud compute addresses list`.
- **HA VPN + Cloud Router (region `us-west1`)**: the GCP end of the tunnel. The Cloud Router talks BGP using ASN `65534`.
- **Cloud DNS `169.254.169.254`**: GCP's reception desk, used by every GCP VM by default.
- **Private zone `gcpcloud.internal`**: GCP's phone book.
- **Forwarding zone**: GCP's note on the desk: "questions about `azurecloud.internal`, send to Azure at `10.10.50.4`."

The tunnel in the middle

The VPN carries all traffic between `10.10.0.0/16` and `10.20.0.0/16`, including DNS. BGP is how each side learns which IP ranges are on the other side, so it knows to send packets into the tunnel.

3. Azure VM looks up a GCP name

What happens when the Azure VM runs `nslookup private.gcpccloud.internal`:

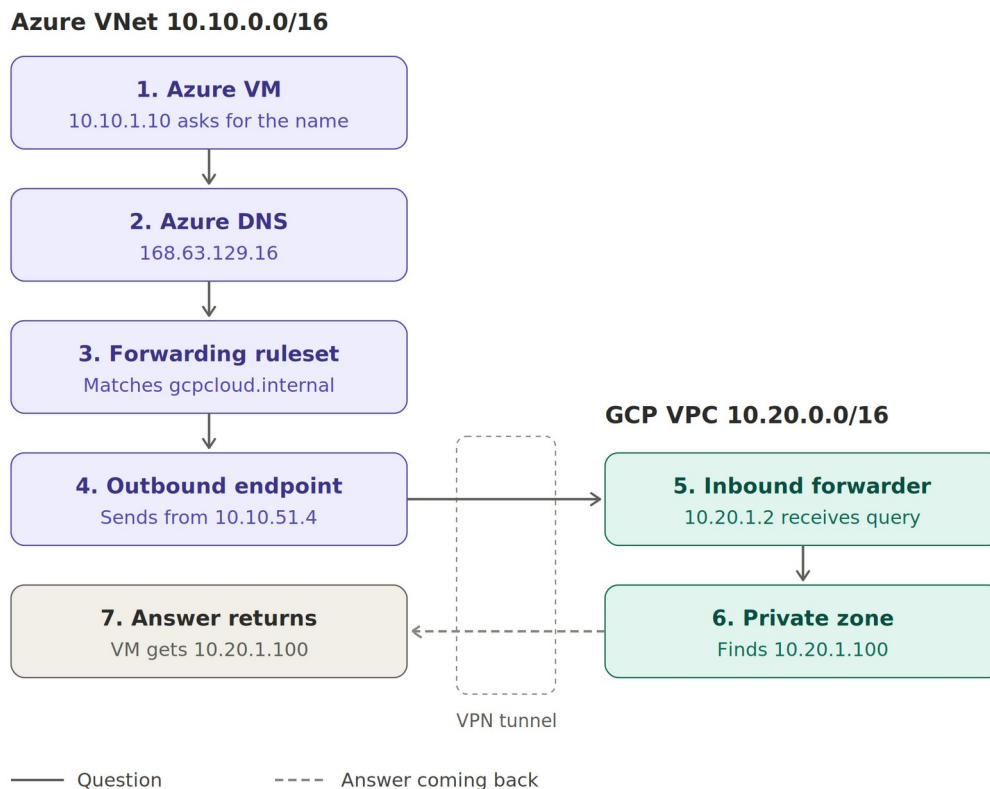


Diagram 2: Azure to GCP name resolution

1. **The Azure VM asks.** The app on `10.10.1.10` wants to reach `private.gcpccloud.internal`, so the VM sends the question to its reception desk, `168.63.129.16`.
2. **Azure DNS checks its notes.** It looks at the forwarding ruleset linked to this VNet.
3. **The rule matches.** The name ends in `gcpccloud.internal`, so the rule says "send this to GCP at `10.20.1.2 / 10.20.2.2`."
4. **The question leaves from the outbound subnet.** Azure sends it from the outbound endpoint (`10.10.51.4`). Azure knows how to reach `10.20.x.x` because GCP's Cloud Router told it via BGP, so the packet goes into the tunnel.
5. **GCP's visitor line picks up.** The inbound forwarder `10.20.1.2` (in `us-west1`, next to the VPN) receives the question as if it came from inside GCP.
6. **GCP checks its phone book.** Cloud DNS finds `private.gcpccloud.internal` → `10.20.1.100`.
7. **The answer goes back.** GCP replies to `10.10.51.4`, which it can reach because Azure advertised `10.10.0.0/16` via BGP. Azure DNS hands the answer to the VM, and the VM connects to `10.20.1.100`.

4. GCP VM looks up an Azure name

What happens when the GCP VM runs `nslookup public.azurecloud.internal`:

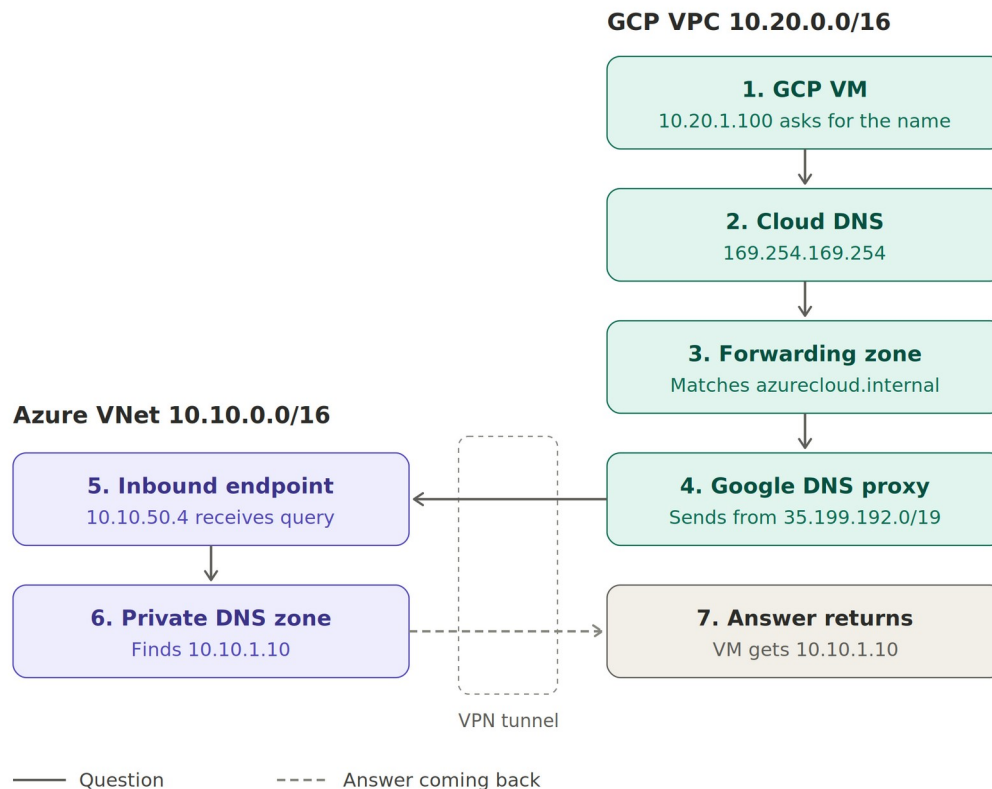


Diagram 3: GCP to Azure name resolution

1. **The GCP VM asks.** The app on `10.20.1.100` wants `public.azurecloud.internal` and asks its reception desk, `169.254.169.254`.
2. **Cloud DNS checks its zones.** It looks at the zones attached to `gcp-vpc`.
3. **The forwarding zone matches.** The name ends in `azurecloud.internal`, so the forwarding zone says "send this to Azure at `10.10.50.4`."
4. **Google sends it from a special range.** GCP does **not** send the question from your VM's IP. It sends it from Google's own DNS range, `35.199.192.0/19`. The packet goes into the tunnel because GCP learned `10.10.0.0/16` from Azure via BGP.
5. **Azure's visitor line picks up.** The inbound endpoint `10.10.50.4` receives the question.
6. **Azure checks its phone book.** The private zone linked to the VNet has `public.azurecloud.internal → 10.10.1.10`.
7. **The answer goes back.** Azure replies to the `35.199.x.x` address. Azure only knows to send that reply into the tunnel because the Cloud Router **advertises** `35.199.192.0/19` over BGP. Without it, the reply would head toward the internet and be lost.

5. What the network layer must provide

DNS forwarding only works if both directions have routes, and BGP supplies all of them:

Traffic	Destination	Who advertises the route
Azure query to GCP	GCP subnets (inbound forwarder IPs 10.20.1.2 and 10.20.2.2)	GCP Cloud Router → Azure
GCP reply to Azure	Azure VNet (outbound endpoint IP 10.10.51.4)	Azure VPN gateway → GCP
GCP query to Azure	Azure VNet (inbound endpoint IP 10.10.50.4)	Azure VPN gateway → GCP
Azure reply to GCP	35.199.192.0/19	GCP Cloud Router (custom advertisement) → Azure

The last row is the one people most often forget. Without the custom advertisement of [35.199.192.0/19](#), GCP's questions reach Azure, but Azure's answers never come back.

The GCP firewall rule that allows [10.10.0.0/16](#) covers Azure's outbound subnet [10.10.51.0/28](#), so Azure's questions are let in.

6. Global routing across GCP regions

A GCP VPC is **global**: one VPC can have subnets in many regions. But the **Cloud Router** is **regional**. Ours lives in `us-west1`, next to the HA VPN and the subnet `10.20.1.0/24`. So what happens to the subnet `10.20.2.0/24` in `us-east1`, where there is no VPN?

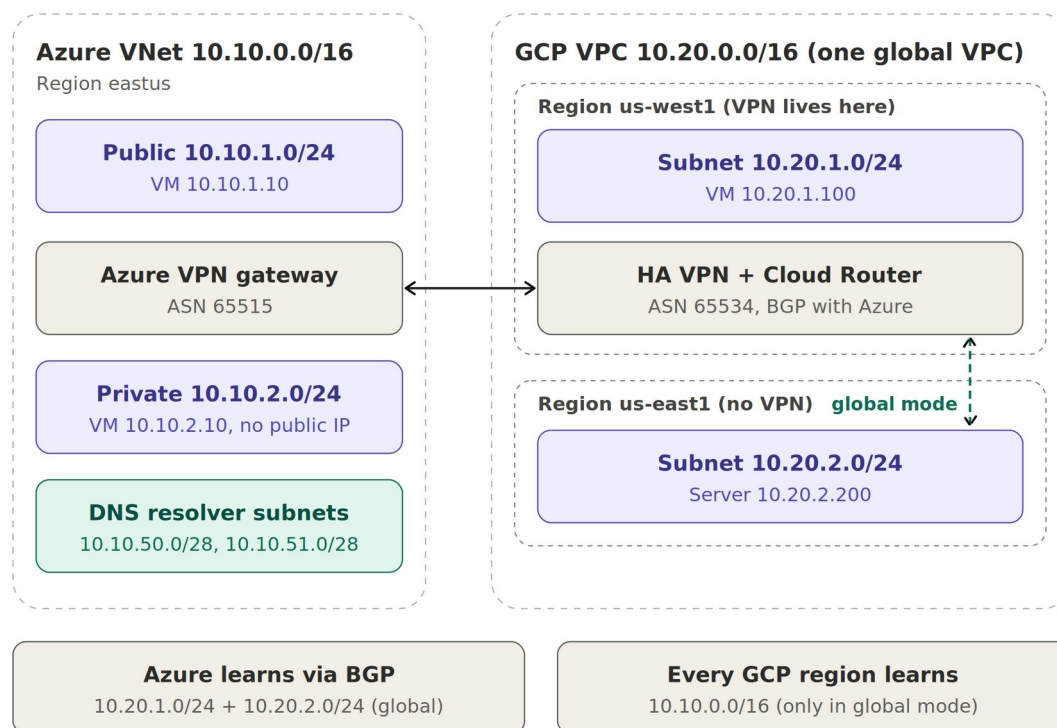


Diagram 4: One VPN in us-west1 serving a second GCP region (us-east1)

The answer depends on one VPC setting called the **dynamic routing mode**:

- **Regional mode (default):** the Cloud Router only works for its own region. It tells Azure about `us-west1` subnets only (`10.20.1.0/24`), and the Azure routes it learns are only given to `us-west1` VMs. The `us-east1` subnet `10.20.2.0/24` is invisible to Azure, and its VMs cannot reach Azure.
- **Global mode:** the Cloud Router works for the whole VPC. It tells Azure about subnets in **all** regions (`10.20.1.0/24` and `10.20.2.0/24`), and gives the Azure routes to VMs in **every** region. Traffic from `us-east1` travels over Google's private backbone to `us-west1`, then through the VPN to Azure.

Behaviour	Regional mode (default)	Global mode
Subnets advertised to Azure	Only <code>us-west1</code> : <code>10.20.1.0/24</code>	All regions: also <code>10.20.2.0/24</code> (<code>us-east1</code>)
Routes learned from Azure (<code>10.10.0.0/16</code>)	Used only by VMs in <code>us-west1</code>	Used by VMs in every region
<code>us-east1</code> server <code>10.20.2.200</code> reaches Azure <code>10.10.2.10</code>	No	Yes, through the <code>us-west1</code> tunnels
Azure reaches <code>10.20.2.200</code>	No	Yes

Behaviour	Regional mode (default)	Global mode
us-east1 VM resolves azurecloud.internal	Fails (no route to 10.10.50.4)	Works
Azure can use forwarder IP 10.20.2.2	No	Yes

How a packet travels from us-east1 to Azure

1. The server 10.20.2.200 in us-east1 sends a packet to 10.10.2.10.
2. Its VPC route table has 10.10.0.0/16 (learned by the Cloud Router in us-west1, shared by global mode). The route points to the VPN tunnels.
3. Google carries the packet across its backbone from us-east1 to us-west1. You don't configure this; it is automatic inside one VPC.
4. The packet enters the HA VPN tunnel in us-west1 and arrives at the Azure VPN gateway.
5. Azure delivers it to 10.10.2.10 in the private subnet. The reply goes back the same way, because Azure learned 10.20.2.0/24 via BGP.

What changes on the Azure side

Nothing extra is needed for 10.10.2.0/24. The Azure VPN gateway advertises the **whole VNet address space** (10.10.0.0/16), which already includes the new subnet. VMs in the private subnet also use Azure DNS by default, and the forwarding ruleset is linked to the whole VNet, so they resolve gcpcloud.internal names immediately.

Being a **private subnet** only removes default outbound internet access. It does not affect traffic over the VPN.

What changes for DNS

- Cloud DNS zones belong to the VPC, not a region, so us-east1 VMs can resolve gcpcloud.internal names in either mode.
- To resolve azurecloud.internal, Cloud DNS forwards to 10.10.50.4. From us-east1, that only works in **global mode**, because the route to 10.10.0.0/16 must exist in that region. From us-west1 (the VPN region), it works in both modes.
- The forwarder IP 10.20.2.2 lives in the us-east1 subnet. Azure can only reach it in global mode, so in regional mode the Azure rule should use 10.20.1.2 only.
- GCP firewall rules are global, so the existing rule allowing 10.10.0.0/16 already covers us-east1.

Trade-off to keep in mind

Azure is in `eastus`, but the VPN is in `us-west1`. So traffic from `us-east1` crosses the country twice: east coast → west coast (Google backbone) → back east to Azure. For production with heavy traffic from `us-east1`, you can add a second HA VPN and Cloud Router in `us-east1` for a much shorter path and extra redundancy. BGP will prefer the closer path automatically.

7. Why each VM must use its default DNS

Both flows start with the VM asking its cloud's built-in DNS (`168.63.129.16` on Azure, `169.254.169.254` on GCP). If a VM uses any other DNS server, it never reaches the ruleset or forwarding zone, so cross-cloud names fail.

That is also what happened with the earlier `.local` zones. Inside Linux, `systemd-resolved` refused to pass `.local` names to `168.63.129.16` at all, so step 1 of the Azure → GCP flow never happened. With `.internal`, the query flows normally from the VM into Azure DNS, and everything in the diagrams takes over from there.

8. Troubleshooting steps

Step 1: Test in halves

Always start here. It tells you which half is broken. On an Azure VM, ask Azure DNS directly (this skips the VM's own resolver):

```
nslookup private.gcpccloud.internal 168.63.129.16
```

Then ask GCP's inbound forwarder directly (this skips Azure DNS and the ruleset):

```
nslookup private.gcpccloud.internal 10.20.1.2
```

- **Normal lookup fails, but 168.63.129.16 works:** the problem is inside the VM. Go to Step 3.
- **168.63.129.16 fails, but 10.20.1.2 works:** the problem is Azure's ruleset. Go to Step 4.
- **Both fail:** the problem is the network or GCP. Go to Step 2, then Step 5.

On a GCP VM, do the same with Azure's inbound endpoint:

```
nslookup public.azurecloud.internal 10.10.50.4
```

Step 2: Check the tunnel and BGP

GCP tunnel status (expect **ESTABLISHED**):

```
gcloud compute vpn-tunnels describe tunnel-0 --region=us-west1 --format="value(status)"
```

GCP BGP status (expect peers **UP** and learned route **10.10.0.0/16**):

```
gcloud compute routers get-status gcp-router --region=us-west1
```

Azure connection and BGP status (expect **Connected**):

```
az network vpn-connection show --resource-group rg-s2s --name conn-gcp-0 --query  
connectionStatus -o tsv  
az network vnet-gateway list-bgp-peer-status --resource-group rg-s2s --name az-vpngw -o  
table
```

Azure learned routes (expect **10.20.0.0/16** subnets and **35.199.192.0/19**):

```
az network vnet-gateway list-learned-routes --resource-group rg-s2s --name az-vpngw -o  
table
```

If **35.199.192.0/19** is missing, re-apply the custom advertisement:

```
gcloud compute routers update gcp-router --region=us-west1 --advertisement-mode=custom  
--set-advertisement-groups=all_subnets --set-advertisement-ranges=35.199.192.0/19
```

If the inbound forwarder IPs are in more than one GCP region, make sure Azure can reach all of them:

```
gcloud compute networks update gcp-vpc --bgp-routing-mode=global
```

Step 3: The VM is not using its default DNS

See which DNS server the Linux VM uses (it should show **168.63.129.16** on Azure):

```
resolvectl status  
ls -l /etc/resolv.conf
```

Check for custom DNS on the Azure VNet and NIC (both should be empty):

```
az network vnet show --resource-group rg-s2s --name az-vnet --query  
dhcpOptions.dnsServers  
az network nic show --resource-group rg-s2s --name <NIC_NAME> --query  
dnsSettings.dnsServers
```

Clear them if needed, then restart the VM:

```
az network vnet update --resource-group rg-s2s --name az-vnet --dns-servers ""
az network nic update --resource-group rg-s2s --name <NIC_NAME> --dns-servers ""
az vm restart --resource-group rg-s2s --name <VM_NAME>
```

If `/etc/resolv.conf` is a plain file instead of a link, restore the normal setup:

```
sudo ln -sf /run/systemd/resolve/stub-resolv.conf /etc/resolv.conf
sudo systemctl restart systemd-resolved
```

Also check for DNS servers hard-coded in netplan:

```
grep -r nameservers -A3 /etc/netplan/
```

Step 4: Azure → GCP fails

Is the ruleset linked to the VNet the VM is in?

```
az dns-resolver vnet-link list --resource-group rg-s2s --ruleset-name az-ruleset -o table
```

Is the rule correct? Check that the domain is `gcpcloud.internal.` (with the trailing dot), the state is `Enabled`, and the target IPs match GCP's inbound forwarder IPs:

```
az dns-resolver forwarding-rule show --resource-group rg-s2s --ruleset-name az-ruleset --name to-gcp-internal
gcloud compute addresses list --filter='purpose="DNS_RESOLVER"' --format="table(address,region,subnetwork)"
```

Can the Azure VM reach port 53 on GCP?

```
nc -vz 10.20.1.2 53
```

Is the inbound policy active, and does the record exist?

```
gcloud dns policies describe allow-inbound-dns
gcloud dns record-sets list --zone=gcpcloud-internal
```

Azure DNS briefly caches failed lookups, so wait a few minutes after a fix before retesting.

Step 5: GCP → Azure fails

Is the forwarding zone pointing to `10.10.50.4` as a **private** target?

```
gcloud dns managed-zones describe to-azure-internal
```

Is the Azure inbound endpoint IP really `10.10.50.4`?

```
az dns-resolver inbound-endpoint show --resource-group rg-s2s --dns-resolver-name az-dns-resolver --name inbound-ep --query "ipConfigurations[0].privateIpAddress" -o tsv
```

Is the Azure private zone linked to the VNet, and does the record exist?

```
az network private-dns link vnet list --resource-group rg-s2s --zone-name azurecloud.internal -o table
az network private-dns record-set a list --resource-group rg-s2s --zone-name azurecloud.internal -o table
```

If all of these look right, the usual cause is a missing `35.199.192.0/19` route in Azure (see Step 2).

Step 6: See whether queries reach GCP at all

Turn on DNS logging for the inbound policy, run the lookup from Azure again, then read the logs:

```
gcloud dns policies update allow-inbound-dns --enable-logging
gcloud logging read 'resource.type="dns_query"' --limit=20 --
format="table(timestamp,jsonPayload.queryName,jsonPayload.sourceIP,jsonPayload.response
Code)"
```

- **No entries:** the query never reached GCP. Check Azure (Steps 3 and 4) or routing (Step 2).
- **NXDOMAIN:** the query arrived but the name was not found. Check the record and zone name.
- **NOERROR:** GCP answered, so the reply is being lost on the way back. Check that GCP learned `10.10.0.0/16` (Step 2).

Step 7: SERVFAIL from 127.0.0.53 on a .local name

If a zone ends in `.local`, Linux `systemd-resolved` treats it as multicast DNS and never sends it to Azure DNS. You will see **SERVFAIL** from `127.0.0.53`, even though `nslookup <name> 168.63.129.16` works. The best fix is to rename the zone to `.internal`. As a quick workaround on one VM:

```
sudo resolvectl domain eth0 '~local'
```

9. Commands

Azure: resolver subnets and endpoints

Get the VNet ID (use it as **<VNET_ID>** below):

```
az network vnet show --resource-group rg-s2s --name az-vnet --query id -o tsv
```

Create the inbound subnet:

```
az network vnet subnet create --resource-group rg-s2s --vnet-name az-vnet --name snet-dns-inbound --address-prefixes 10.10.50.0/28 --delegations Microsoft.Network/dnsResolvers
```

Create the outbound subnet:

```
az network vnet subnet create --resource-group rg-s2s --vnet-name az-vnet --name snet-dns-outbound --address-prefixes 10.10.51.0/28 --delegations Microsoft.Network/dnsResolvers
```

Create the DNS Private Resolver:

```
az dns-resolver create --resource-group rg-s2s --name az-dns-resolver --location eastus --id "<VNET_ID>"
```

Create the inbound endpoint with the fixed IP 10.10.50.4:

```
az dns-resolver inbound-endpoint create --resource-group rg-s2s --dns-resolver-name az-dns-resolver --name inbound-ep --location eastus --ip-configurations "[{private-ip-address:'10.10.50.4',private-ip-allocation-method:'Static',id:'<VNET_ID>/subnets/snet-dns-inbound'}]"
```

Create the outbound endpoint:

```
az dns-resolver outbound-endpoint create --resource-group rg-s2s --dns-resolver-name az-dns-resolver --name outbound-ep --location eastus --id "<VNET_ID>/subnets/snet-dns-outbound"
```

Azure: private zone, ruleset and rule

```
az network private-dns zone create --resource-group rg-s2s --name azurecloud.internal

az network private-dns link vnet create --resource-group rg-s2s --zone-name azurecloud.internal --name link-az-vnet --virtual-network az-vnet --registration-enabled false

az network private-dns record-set a add-record --resource-group rg-s2s --zone-name azurecloud.internal --record-set-name public --ipv4-address 10.10.1.10
```

Get the outbound endpoint ID (use it as **<OUTBOUND_EP_ID>**):

```
az dns-resolver outbound-endpoint show --resource-group rg-s2s --dns-resolver-name az-dns-resolver --name outbound-ep --query id -o tsv

az dns-resolver forwarding-ruleset create --resource-group rg-s2s --name az-ruleset --location eastus --outbound-endpoints "[{id:'<OUTBOUND_EP_ID>'}]"

az dns-resolver forwarding-rule create --resource-group rg-s2s --ruleset-name az-ruleset --name to-gcp-internal --domain-name "gcpcloud.internal." --forwarding-rule-state Enabled --target-dns-servers "[{ip-address:'10.20.1.2',port:53},{ip-address:'10.20.2.2',port:53}]"

az dns-resolver vnet-link create --resource-group rg-s2s --ruleset-name az-ruleset --name link-az-vnet --id "<VNET_ID>"
```

GCP: private zone, inbound policy, forwarding zone, route advertisement

```
gcloud dns managed-zones create gcpcloud-internal --dns-name="gcpcloud.internal." --visibility=private --networks=gcp-vpc --description="GCP private zone"
```

```
gcloud dns record-sets create private.gcpcloud.internal. --zone=gcpcloud-internal --
type=A --ttl=300 --rrdatas=10.20.1.100

gcloud dns record-sets create public.gcpcloud.internal. --zone=gcpcloud-internal --
type=A --ttl=300 --rrdatas=10.20.2.200

gcloud dns policies create allow-inbound-dns --networks=gcp-vpc --enable-inbound-
forwarding --description="Allow Azure to query Cloud DNS"
```

Get the inbound forwarder IPs (use these in the Azure forwarding rule):

```
gcloud compute addresses list --filter='purpose="DNS_RESOLVER"' --
format="table(address,region,subnetwork)"

gcloud dns managed-zones create to-azure-internal --dns-name="azurecloud.internal." --
visibility=private --networks=gcp-vpc --private-forwarding-targets=10.10.50.4 --
description="Forward to Azure Private Resolver"

gcloud compute routers update gcp-router --region=us-west1 --advertisement-mode=custom
--set-advertisement-groups=all_subnets --set-advertisement-ranges=35.199.192.0/19
```

New subnets and global routing

Azure: create the private subnet (no default outbound internet access):

```
az network vnet subnet create --resource-group rg-s2s --vnet-name az-vnet --name snet-
private --address-prefixes 10.10.2.0/24 --default-outbound false

az network private-dns record-set a add-record --resource-group rg-s2s --zone-name
azurecloud.internal --record-set-name private-vm --ipv4-address 10.10.2.10
```

GCP: create the two subnets, one per region:

```
gcloud compute networks subnets create gcp-subnet-west --network=gcp-vpc --region=us-
west1 --range=10.20.1.0/24

gcloud compute networks subnets create gcp-subnet-east --network=gcp-vpc --region=us-
east1 --range=10.20.2.0/24
```

GCP: turn on global routing, then confirm it says **GLOBAL**:

```
gcloud compute networks update gcp-vpc --bgp-routing-mode=global

gcloud compute networks describe gcp-vpc --format="value(routingConfig.routingMode)"
```

Check the inbound forwarder IPs (one per subnet: **10.20.1.x** in **us-west1**, **10.20.2.x** in **us-east1**):

```
gcloud compute addresses list --filter='purpose="DNS_RESOLVER"' --
format="table(address,region,subnetwork)"
```

Confirm Azure now learns both **10.20.1.0/24** and **10.20.2.0/24**:

```
az network vnet-gateway list-learned-routes --resource-group rg-s2s --name az-vpngw -o
table
```

From a VM in **us-east1** (the region without the VPN):

```
nslookup private-vm.azurecloud.internal

ping 10.10.2.10
```

Verify

```
az network vnet-gateway list-learned-routes --resource-group rg-s2s --name az-vpngw -o
table

gcloud compute routers get-status gcp-router --region=us-west1
```

From an Azure VM:

```
nslookup private.gcpcloud.internal
```

From a GCP VM:

```
nslookup public.azurecloud.internal
```

10. Lessons learned

- **Each VM must use its cloud's built-in DNS** (168.63.129.16 on Azure, 169.254.169.254 on GCP). If a VM uses any other DNS server, it never reaches the ruleset or forwarding zone, so cross-cloud names fail.
- **Avoid .local for private zones.** On Linux, `systemd-resolved` (127.0.0.53) reserves `.local` for multicast DNS and never forwards those names to Azure DNS, which causes `SERVFAIL`. Use `.internal`, which is officially reserved for private networks.
- **Advertise 35.199.192.0/19 from the Cloud Router.** Without it, Azure cannot send answers back to GCP's forwarded queries.
- **Use `--private-forwarding-targets`** on the GCP forwarding zone so queries go through the VPN, not the internet.
- **Turn on global routing for multi-region GCP.** With one VPN in `us-west1`, subnets in other regions (like `10.20.2.0/24` in `us-east1`) can only reach Azure, and resolve Azure names, when the VPC uses `--bgp-routing-mode=global`.
- **Test in halves.** `nslookup <name> 168.63.129.16` on an Azure VM bypasses the VM's own resolver and shows whether the cloud side is working.